Part 3: Future Work — Heterogeneous Multi-Core CIMP and Quantization Co-Design

Project: ZigZag-CIMP Extension
Continuation of: Part 1 (Energy Model) and Part 2 (ZigZag Integration)

---

1. The Problem Restated

Part 2 identified a fundamental tension in CapRAM accelerator design: no single array size works well across all layers of a DNN. The 128 × 8192 array achieved less than 1% spatial utilization on ResNet-18 Conv1 (3 input channels, 64 output channels), resulting in a 527× energy gap compared to an ideally-fitting workload. Deeper layers with hundreds of channels would utilize the array much better, but early layers are pathologically mismatched.

This is not unique to CapRAM. Every fixed-size IMC array faces the same problem. But CapRAM amplifies it because the device-level efficiency is so high that any utilization loss is proportionally more wasteful — the array is "too good" to sit idle.

Two complementary strategies address this from opposite ends of the stack: heterogeneous multi-core architectures (a hardware solution) and accumulator-aware quantization (an algorithm solution). Both are directly implementable within the ZigZag ecosystem, and their combination represents the most promising path toward closing the peak-to-system gap identified in Part 2.

---

2. Heterogeneous Multi-Core CIMP via Stream

2.1 What Stream Is

Stream is a design space exploration framework developed at KU Leuven (MICAS group) that extends ZigZag from single-core to multi-core heterogeneous dataflow accelerators. Where ZigZag answers "how should I map one layer onto one core?", Stream answers "how should I schedule an entire network across multiple cores that may have different sizes, precisions, and memory hierarchies?"

Stream supports three scheduling paradigms:

Layer-by-layer (coarse-grained): Each layer is fully computed on one core before the next layer begins. This is the traditional approach and what the current ZigZag-CIMP integration implements. It maximizes data reuse within a layer but incurs costly off-chip memory accesses between layers.

Layer-fused (fine-grained): Parts of adjacent layers are computed together, with intermediate activations staying on-chip. A small tile of Conv1's output is immediately consumed by Conv2

before the next Conv1 tile is computed. This reduces off-chip memory traffic dramatically but requires careful scheduling to avoid deadlocks.

Pipelined (batched): Different layers run on different cores simultaneously, each processing a different input from a batch. This maximizes throughput but requires batched inputs, which is unsuitable for latency-critical edge applications.

Stream implements a constraint-optimization engine that jointly optimizes core assignment, scheduling order, and data placement across all layers and cores. It includes COALA, a validated communication and memory-aware latency/energy model that accounts for inter-core data transfers.

The key result from the Stream paper: combining layer fusion with heterogeneous multi-core architectures yields up to 2.2× lower energy-delay product compared to homogeneous layer-by-layer execution, and up to 30× EDP reduction compared to naive single-core scheduling.

2.2 Why Heterogeneous CIMP

The utilization problem from Part 2 has an elegant architectural solution: instead of one large array, use multiple arrays of different sizes, each optimized for a different class of layers.

Consider a ResNet-18 accelerator with three CIMP cores:

Core	Array Size	Target Layers	Rationale
Core A	64 × 256	Conv1, early layers (C = 3–64)	Small row count matches few input channels; moderate columns for 64 output channels
Core B	256 × 1024	Middle layers (C = 64–256)	Balanced dimensions for typical mid-network layers
Core C	128 × 8192	FC layers, attention heads	Large row count for high-dimensional dot products

Each core would have its own CIMP physics parameters (possibly different ON/OFF ratios, ADC resolutions, or even different memory technologies — SONOS for inference-only cores, ferroelectric for cores that need retraining). Stream would schedule the network across these cores, fusing layers where possible to keep intermediate activations on-chip.

The ZigZag-CIMP integration from Part 2 already provides the per-core energy and latency model. Stream would consume these as inputs and optimize the inter-core scheduling. The CIMP-specific modifications needed are minimal: Stream already supports heterogeneous cores with different energy/latency characteristics — it just needs to receive CIMP-parameterized ZigZag cores instead of SRAM-parameterized ones.

2.3 What Heterogeneous CIMP Would Solve

Spatial utilization: By assigning Conv1 to Core A (64 × 256), the spatial utilization jumps from

<1% (on the 128 × 8192 array) to roughly 64 × 147 / (64 × 256) = 57%. Not perfect, but 57× better than before.

Temporal overhead: With layer fusion, Conv1's output tiles can be immediately consumed by Conv2 on Core B without writing to DRAM. This eliminates the six-level temporal loop nest that dominated Conv1's latency in Part 2 (8.6 million ns) and replaces it with a pipelined execution where each core processes its tile and passes the result forward.

Memory wall: The dominant energy contributor in Part 2 was SRAM/DRAM data movement. Layer fusion keeps intermediate activations in small on-chip buffers between adjacent cores, bypassing the SRAM entirely for fused layer pairs. Stream's COALA model quantifies this: inter-core transfer energy is typically 10–100× cheaper than SRAM access, which is itself 10× cheaper than DRAM.

EDP improvement estimate: Based on Stream's published results (2.2× EDP reduction from heterogeneous fusion on conventional accelerators), and considering that CapRAM's extreme compute-to-memory ratio would amplify the benefit of reduced memory traffic, a conservative estimate for heterogeneous CIMP is 3–5× EDP improvement over the single-core results from Part 2. This would bring the system TOPS/W from 2.06 (Part 2, Conv1) to roughly 6–10 TOPS/W — still below the macro peak of 107 TOPS/W, but a major step toward closing the gap.

2.4 Implementation Path

The integration path from ZigZag-CIMP to Stream-CIMP involves:

1. **Define multiple CIMP cores in Stream's hardware YAML.** Each core is a ZigZag-CIMP accelerator with its own array dimensions, ON/OFF ratio, and ADC resolution. Stream's YAML format supports this natively.
2. **Define the inter-core communication fabric.** Stream models a network-on-chip (NoC) or shared memory bus connecting cores. For CIMP, this could be a simple shared SRAM with known read/write costs, or a direct core-to-core link if 3D stacking allows vertical data passing.
3. **Run Stream's constraint optimizer.** Stream assigns each layer (or layer tile) to a core and determines the scheduling order that minimizes total EDP, latency, or energy under memory capacity constraints.
4. **Analyze the result.** Stream produces a per-layer, per-core execution trace showing utilization, energy, and data movement — exactly the information needed to iterate on core sizing and memory placement.

The CIMP physics engine (Part 1) does not need modification — it already parameterizes per-core energy and latency. The ZigZag-CIMP integration (Part 2) provides the per-core model that Stream consumes. The only new work is the Stream-level hardware description and the inter-core communication model.

---

3. Quantization-Aware Training for ADC Resolution Reduction

3.1 The ADC Problem

Part 1 established that ADC energy is the dominant macro-level cost at small array sizes, contributing 70–80% of total compute energy via the Walden FOM. Part 2 showed that at the system level, memory costs dilute this dominance but ADC optimization still yields meaningful savings. The question is: how low can the ADC resolution go before inference accuracy degrades?

Paper 2 answered this with the A2Q+ framework (Colbert et al., ICML 2024), demonstrating that the required ADC resolution for a W4A8 configuration on ResNet-18 can be reduced from 19 bits (naive) to 12 bits (with input bit-slicing) to 7–9 bits (with slice-wise A2Q+ training) — without accuracy degradation.

3.2 How A2Q+ Works

The core idea is to constrain the ℓ_1-norm of quantized weight vectors during training so that the worst-case dot product never exceeds the ADC's representable range. This is the full-precision guarantee (FPG): every valid input–weight combination maps to a unique digital code, preventing overflow.

A2Q+ improves on the original A2Q formulation in two ways:

Tighter bound via zero-centering. By enforcing that the quantized weight vector sums to zero ($\Sigma[w]_i = 0$), A2Q+ allows the accumulator's full dynamic range to be utilized regardless of input sign. This yields an ℓ_1-norm budget up to 4× larger than A2Q's conservative bound, significantly alleviating pressure on model weights and preserving expressivity.

Slice-wise constraints. Paper 2 extends A2Q+ to operate on individual weight bit slices. Each slice [w]^(j) is independently projected onto a feasible set defined by a per-slice ℓ_1-norm budget. This allows the ADC resolution to be matched to the per-slice accumulation range rather than the full-precision weight range, enabling much lower B_y values.

The practical result (Paper 2, Figure 4): with S_w = 4 and B_y = 8, an effective allocation of only ~2 bits per weight is sufficient to match the full-precision baseline on CIFAR-10/ResNet-18. Weight sparsity emerges naturally from the ℓ_1-norm constraint — A2Q+ inherently prunes weights that would cause accumulator overflow.

3.3 Impact on CIMP System Design

The energy model from Part 1 shows that reducing B_y from 14 to 12 bits yields roughly 8× energy reduction at the macro level (Schreier FOM 2^(2B_y) scaling). Each additional bit saved yields another 4× reduction. At the system level, the impact is more modest because memory costs dominate, but the latency benefit is direct: fewer ADC bits means faster conversion, which reduces the stall time between compute and digitization.

For heterogeneous CIMP specifically, A2Q+ enables a powerful optimization: different cores can use different ADC resolutions. Core A (small array, Conv1) might need only 7-bit ADC because

its small K produces limited column sums. Core C (large array, FC layers) might need 10-bit ADC due to its long columns. Training with slice-wise A2Q+ can enforce per-layer overflow avoidance guarantees that match each core's hardware capabilities.

3.4 Integration with ZigZag-CIMP

The current ZigZag-CIMP implementation already accepts `adc_resolution` as a YAML parameter. To integrate A2Q+:

1. **Train the target model with A2Q+** using the Brevitas quantization library, specifying per-layer accumulator bitwidth targets that match each CIMP core's ADC resolution.
2. **Export the quantized model as ONNX.** The quantized weights automatically satisfy the ℓ_1-norm constraints, guaranteeing overflow-free inference on the target hardware.
3. **Run ZigZag-CIMP with the A2Q+-aware ADC resolution** for each core. The energy model uses the reduced B_y to compute lower E_cap and E_ADC, and the latency model uses the reduced N_av (fewer averaging cycles needed when fewer LSB levels must be distinguished).
4. **Verify accuracy.** The A2Q+ training guarantees mathematical overflow avoidance, but the combined effect of quantization noise, CIMP device variation (σ_C,pgr), and kTC readout noise should be validated through hardware-in-the-loop simulation or noise-injected inference.

---

4. The Combined Vision: Stream + A2Q+ + CIMP

The three components — heterogeneous multi-core scheduling (Stream), quantization-aware training (A2Q+), and capacitive device physics (CIMP) — form a co-design loop:

```
                ┌──────────────────────┐
                │   Target DNN (ONNX)  │
                └──────────┬───────────┘
                           │
                ┌──────────▼───────────┐
                │   A2Q+ Training      │
                │   Per-layer B_y      │
                │   Per-slice ℓ₁ norm  │
                └──────────┬───────────┘
                           │
        ┌──────────────────▼──────────────────┐
        │   Stream Scheduler                  │
        │   Layer → Core assignment           │
        │   Layer fusion schedule             │
        │   Inter-core communication          │
        └──────┬──────────────┬───────────────┘
               │              │
   ┌───────────▼──┐   ┌───────▼──────────┐
   │ ZigZag-CIMP │   │  ZigZag-CIMP     │   ...
   │ Core A       │   │  Core B          │
   │ 64 × 256     │   │  256 × 1024      │
   │ B_y = 7      │   │  B_y = 9         │
   │ ON/OFF = 50 │   │  ON/OFF = 100    │
   └──────────────┘   └──────────────────┘
```

The outer loop iterates: A2Q+ produces a quantized model with per-layer accumulator constraints → Stream schedules it across heterogeneous CIMP cores → ZigZag-CIMP evaluates per-core energy/latency → the results inform the next iteration's core sizing and ADC targets.

What this would achieve compared to Part 2's single-core results:

Aspect	Part 2 (Single Core)	Part 3 Vision (Multi-Core + A2Q+)
Spatial utilization (Conv1)	<1%	~57% (matched core)
System TOPS/W (Conv1)	2.06	Est. 6–10 (fusion + matched core)
System TOPS/W (ideal layer)	19.29	~19.29 (unchanged, already ideal)
System energy (Conv1)	114.5M pJ	Est. 20–40M pJ (less data movement)
ADC resolution	Fixed 8-bit all layers	Per-core: 7–10 bits
Memory bottleneck	SRAM/DRAM dominant	Reduced via layer fusion
Network-wide EDP	Single-layer only	Full ONNX optimized

5. Open Questions and Research Directions

5.1 3D Stacking and the Weight Memory Question

Part 2 noted the unresolved question of how 64 vertically stacked CapRAM layers interact with ADC readout. If 3D stacking effectively creates a 1024 × 65,536 array read through a single ADC column, the kTC noise analysis from Part 1 would need to account for the aggregate parasitic capacitance of 64 stacked layers. Alternatively, if each stack layer is read independently (time-multiplexed through the ADC), the latency model changes fundamentally. This architectural decision has cascading effects on the CIMP energy model, the ZigZag integration, and the Stream scheduling — making it the highest-priority question to resolve.

5.2 Noise-Aware Training Beyond Overflow Avoidance

A2Q+ guarantees that the quantized dot product never overflows the ADC range. But it does not account for the analog noise sources unique to CapRAM: kTC noise (addressed by N_{av} averaging), device variation ($\sigma_{C,pgr}$), and programming drift (especially relevant for SONOS-based retention). Paper 2 proposes noise-aware training by injecting Gaussian noise into the dot-product formulation during QAT. Combining this with A2Q+'s overflow constraints would produce models that are simultaneously overflow-safe and noise-tolerant — a requirement for deployment on real CapRAM hardware.

5.3 Transformer Workloads

The analysis in Parts 1–2 focused on ResNet-18 (CNN). Transformer architectures (ViT, BERT, LLMs) have fundamentally different matrix dimensions: attention layers perform large, regular matrix multiplications (e.g., 768 × 768 or 2048 × 2048) that would fill a large CapRAM array much more uniformly than CNN convolutions. Running Stream-CIMP on transformer ONNX models would likely show a dramatically smaller peak-to-system gap, potentially validating CapRAM as a particularly strong fit for transformer inference at the edge.

5.4 Relaxing the Full-Precision Guarantee

Paper 2 operates under the FPG: every dot product must map to a unique ADC code. This is a conservative constraint that drives ADC resolution (and therefore energy) upward. Future work could explore probabilistic relaxations — allowing a small probability of overflow (e.g., 10^{-6} per inference) in exchange for 1–2 fewer ADC bits. The accuracy impact could be measured empirically through Monte Carlo simulation on the quantized model, and the energy savings quantified through the CIMP energy model. This would be especially valuable for large arrays ($K > 1024$) where the current FPG imposes $B_y > 12$ bits.

5.5 Per-Layer Precision Optimization

The current analysis uses fixed W4A4 across all layers. Modern mixed-precision quantization (e.g., HAWQ, HAQ) assigns different bitwidths to different layers based on sensitivity analysis. Combining mixed-precision with heterogeneous CIMP cores would allow each core to be

specialized not just in array size but also in precision: a 2-bit weight core for noise-insensitive layers, a 4-bit core for sensitive layers. The CIMP energy model from Part 1 already parameterizes S_w, so the DSE infrastructure supports this exploration today.

---

6. Summary

The path from device physics (Part 1) through single-core system analysis (Part 2) to heterogeneous multi-core co-design (Part 3) traces a coherent progression from "how good is the device?" to "how good is the system?" to "how do we close the gap?" The answer to the last question is three-fold: (1) use multiple CIMP cores of different sizes to match diverse layer dimensions, scheduled via Stream's layer-fusion optimizer; (2) use A2Q+ quantization-aware training to minimize ADC resolution per core without sacrificing accuracy; and (3) exploit CapRAM's 3D stacking to eliminate the memory wall that currently absorbs 50× of the device-level efficiency. All three directions are directly implementable within the existing ZigZag/Stream ecosystem with the CIMP physics integration already in place.